INSTITUTO SUPERIOR TECNOLÓGICO QUITO

Excelencia en Educación Superior

5.1 Clasificacion

Erick Mejia

In [3]:
```
import sklearn.metrics as metrics
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_predict, cross_val_score, train_test_
from sklearn import preprocessing
from evaluacion_funciones import *
```

In [4]:
```
# Carga de datos.
datos = load_iris()
X = datos.data[:,2:] # Utilizamos solo 2 atributos.
y = datos.target
print(np.shape(X))
```

```
(150, 2)
```

In [5]:
```
# Métricas de evaluación.
metricas = {
  'ACC':    metrics.accuracy_score,
  'PREC':   lambda y_true, y_pred:
            metrics.precision_score(y_true, y_pred, average='micro'),
  'RECALL': lambda y_true, y_pred:
            metrics.recall_score(y_true, y_pred, average='micro'),
  'F1':     lambda y_true, y_pred:
            metrics.f1_score(y_true, y_pred, average='micro')}
```

In [6]:
```
# 1) Partición de datos externa
X_training, X_testing, y_training, y_testing = train_test_split(X, y, test_size=0.2
print(np.shape(X_training))
```

```
(120, 2)
```

In [7]:
```
# ------------- TRAINING ---------------- #
```

In [8]:
```
# 2) Extracción de características
```

In [9]:
```
# 3) Estandarización de los datos de entrenamiento
standardizer = preprocessing.StandardScaler()
X_stdr = standardizer.fit_transform(X_training)
# print(X_stdr)
```

In [10]:
```
# 4) Selección de atributos
```

In [11]:
```
# 5) Construcción del algoritmo de aprendizaje.
algoritmos = {'LOGR': LogisticRegression(penalty='l1', solver='saga', max_iter=1000
```

In [12]:
```
# 5.1) Validación cruzada interna y Optimización de los hiperparámetros
y_pred = {}
for nombre, alg in algoritmos.items():
    y_pred[nombre] = cross_val_predict(alg, X_stdr, y_training, cv=KFold(n_splits=1
    results = evaluacion(y_training, y_pred[nombre], metricas)
    print(metrics.confusion_matrix(y_training, y_pred[nombre]))
    print(results)

#     results = cross_val_score(alg, X_stdr, y_training, cv = KFold(n_splits=10, sh
#     print("Accuracy:   %0.4f +/- %0.4f" % (results.mean(), results.std()))
```

```
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1135: FutureWarning: 'penalty' was deprecated in version 1.8 and will be remo
ved in 1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instea
d of penalty='l1', and C=np.inf instead of penalty=None.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1160: UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty
is deprecated. Please use l1_ratio only.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1135: FutureWarning: 'penalty' was deprecated in version 1.8 and will be remo
ved in 1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instea
d of penalty='l1', and C=np.inf instead of penalty=None.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1160: UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty
is deprecated. Please use l1_ratio only.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1135: FutureWarning: 'penalty' was deprecated in version 1.8 and will be remo
ved in 1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instea
d of penalty='l1', and C=np.inf instead of penalty=None.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1160: UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty
is deprecated. Please use l1_ratio only.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1135: FutureWarning: 'penalty' was deprecated in version 1.8 and will be remo
ved in 1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instea
d of penalty='l1', and C=np.inf instead of penalty=None.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1160: UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty
is deprecated. Please use l1_ratio only.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1135: FutureWarning: 'penalty' was deprecated in version 1.8 and will be remo
ved in 1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instea
d of penalty='l1', and C=np.inf instead of penalty=None.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1160: UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty
is deprecated. Please use l1_ratio only.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1135: FutureWarning: 'penalty' was deprecated in version 1.8 and will be remo
ved in 1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instea
d of penalty='l1', and C=np.inf instead of penalty=None.
  warnings.warn(
```

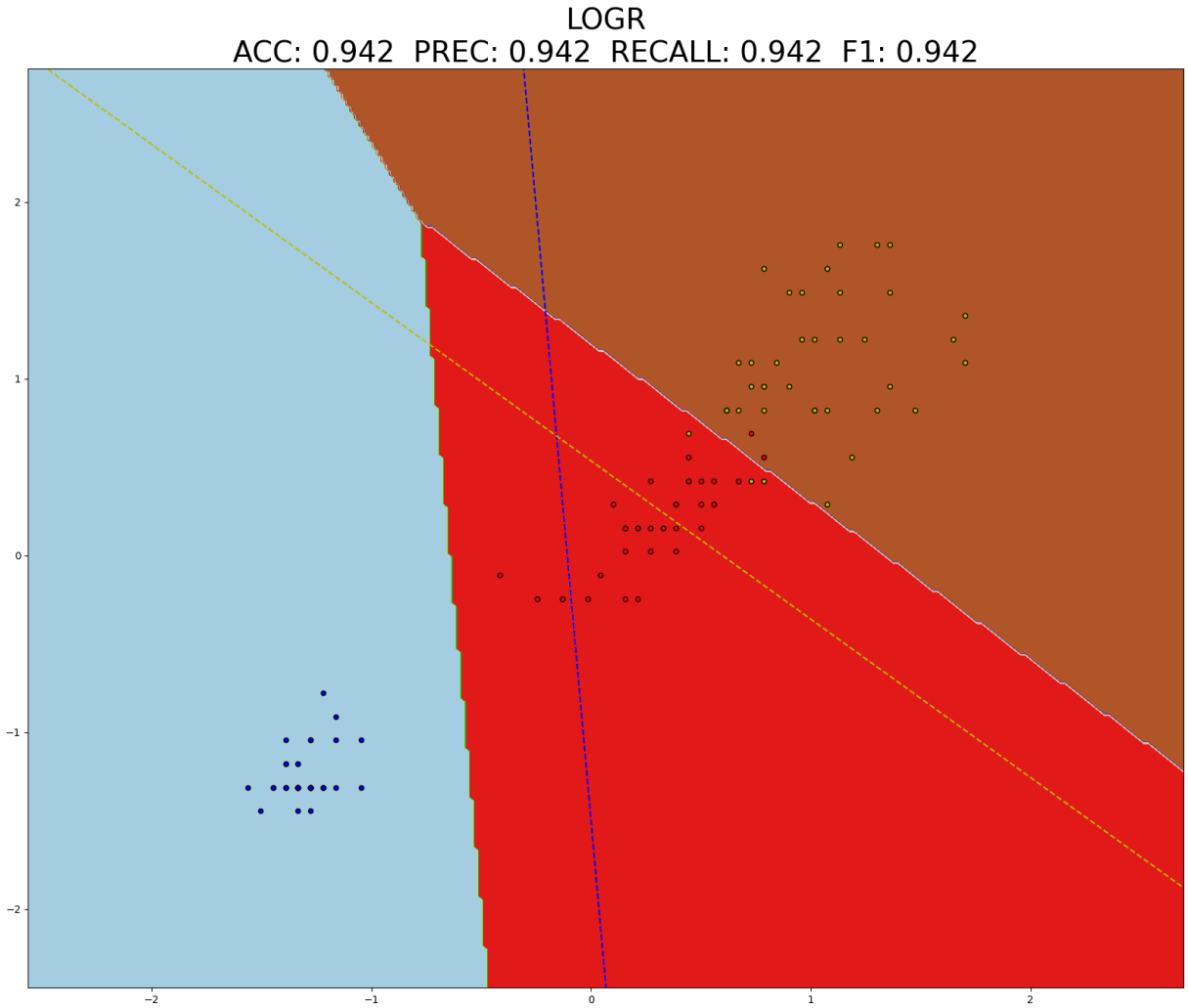
```
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1160: UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty
is deprecated. Please use l1_ratio only.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1135: FutureWarning: 'penalty' was deprecated in version 1.8 and will be remo
ved in 1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instea
d of penalty='l1', and C=np.inf instead of penalty=None.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1160: UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty
is deprecated. Please use l1_ratio only.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1135: FutureWarning: 'penalty' was deprecated in version 1.8 and will be remo
ved in 1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instea
d of penalty='l1', and C=np.inf instead of penalty=None.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1160: UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty
is deprecated. Please use l1_ratio only.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1135: FutureWarning: 'penalty' was deprecated in version 1.8 and will be remo
ved in 1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instea
d of penalty='l1', and C=np.inf instead of penalty=None.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1160: UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty
is deprecated. Please use l1_ratio only.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1135: FutureWarning: 'penalty' was deprecated in version 1.8 and will be remo
ved in 1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instea
d of penalty='l1', and C=np.inf instead of penalty=None.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1160: UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty
is deprecated. Please use l1_ratio only.
  warnings.warn(
[[40  0  0]
 [ 0 38  3]
 [ 0  4 35]]
{'ACC': 0.9416666666666667, 'PREC': 0.9416666666666667, 'RECALL': 0.941666666666666
7, 'F1': 0.9416666666666667}
```

In [13]:

```python
# 5.2) Entrenamiento del modelo definitivo
model = algoritmos['LOGR'].fit(X_stdr, y_training)

# Visualización de las fronteras de decisión
mapa_modelo_clasif_2d(X_stdr, y_training, model, results, nombre)
```

```
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1135: FutureWarning: 'penalty' was deprecated in version 1.8 and will be remo
ved in 1.10. To avoid this warning, leave 'penalty' set to its default value and use
'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instea
d of penalty='l1', and C=np.inf instead of penalty=None.
  warnings.warn(
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\sklearn\linear_model\_logis
tic.py:1160: UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty
is deprecated. Please use l1_ratio only.
  warnings.warn(
C:\Users\erick\Documents\machine-2026\evaluacion_funciones.py:50: UserWarning: No da
ta for colormapping provided via 'c'. Parameters 'cmap' will be ignored
  plt.scatter(X[idx, 0], X[idx, 1], c=color, cmap=plt.cm.Paired,
C:\Users\erick\Documents\machine-2026\evaluacion_funciones.py:62: RuntimeWarning: di
vide by zero encountered in scalar divide
  return (-(x0 * coef[c, 0]) - intercept[c]) / coef[c, 1]
```



In [14]:
```
# ------------- PREDICTION ---------------- #
```

In [15]:
```
# 6) Extracción de las características de test
```

In [16]:
```
# 7) Estandarización de las característiacs de test
X_test_stdr = standardizer.transform(X_testing)
```

In [17]:
```
# 8) Selección de los atributos de test
```

In [18]:
```
# 9) Predicción del conjunto de test
y_pred_test = model.predict(X_test_stdr)
print(y_pred_test)
```

```
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
```

In [19]:
```
# 10) Evaluación del modelo sobre el conjunto de test
results = evaluacion(y_testing, y_pred_test, metricas)
print(results)
print(metrics.confusion_matrix(y_testing, y_pred_test))
```

```
{'ACC': 1.0, 'PREC': 1.0, 'RECALL': 1.0, 'F1': 1.0}
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

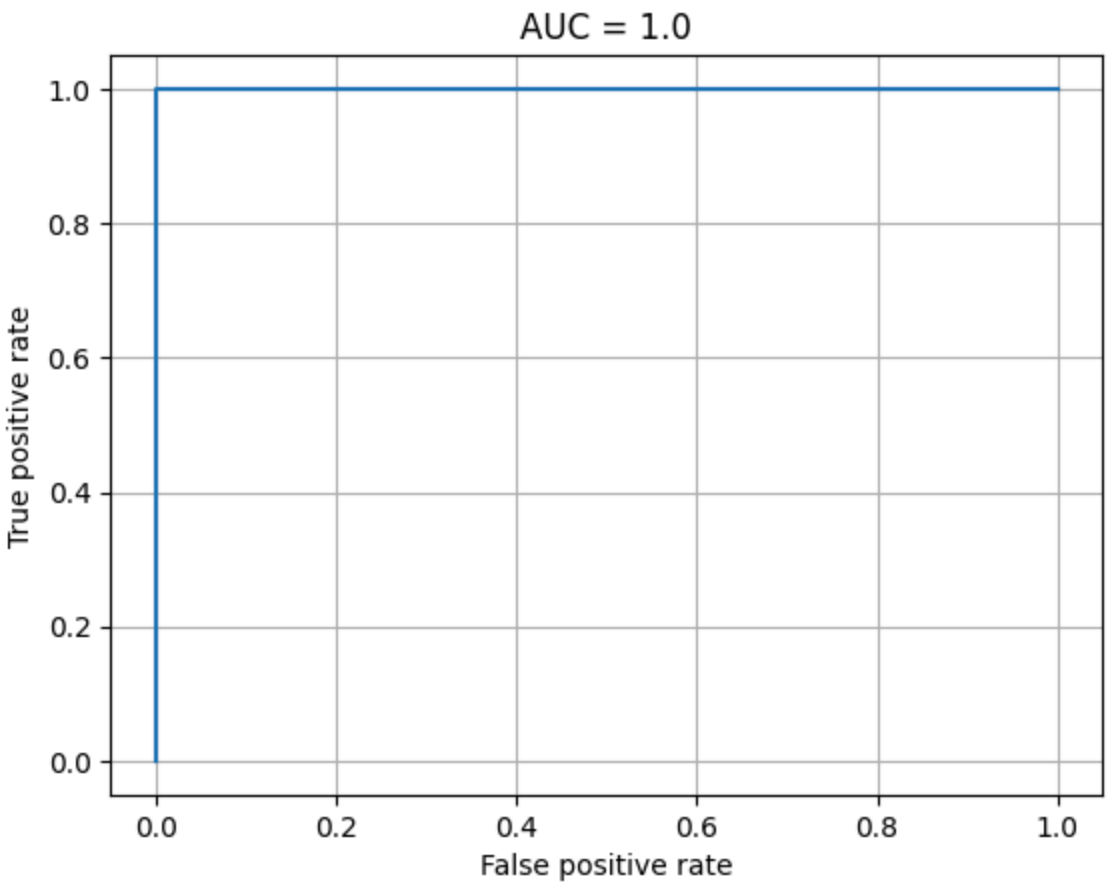
In [20]:
```
# Ploteamos la curva ROC
y_proba_test = model.predict_proba(X_test_stdr) # "predict_proba" para extraer prob

y_test_bin = preprocessing.label_binarize(y_testing, classes=[0,1,2]) # Usar "label

auc = metrics.roc_auc_score(y_testing, y_proba_test, multi_class='ovr') # Area Unde

fpr, tpr, th = metrics.roc_curve(y_test_bin[:,1], y_proba_test[:,1])

plt.plot(fpr, tpr)
plt.xlabel('False positive rate')
plt.ylabel('True positive rate')
plt.title('AUC = ' + str(np.round(auc,4)))
plt.grid()
plt.show()
```



In [21]:
```
import numpy as np  # Sirve para trabajar con números en forma de tabla

print("=== PREDICCIÓN DE FLOR IRIS ===")
print("Introduce los valores:\n")

try:
    # Pedimos los datos al usuario
    petal_length = float(input("Longitud del pétalo (cm): "))
    petal_width = float(input("Ancho del pétalo (cm): "))

    # Guardamos los datos en una forma que el modelo entienda (como una mini tabla)
    sample = np.array([[petal_length, petal_width]])

    # Ajustamos los datos igual que cuando se entrenó el modelo
    sample_scaled = standardizer.transform(sample)

    # El modelo hace la predicción (devuelve 0, 1 o 2)
    prediction = model.predict(sample_scaled)[0]

    # Convertimos ese número en el nombre de la flor
    labels = datos.target_names
    predicted_class = labels[prediction]

    # Calculamos qué tan seguro está el modelo
    probs = model.predict_proba(sample_scaled)[0]

    # Mostramos el resultado
    print("\n-----------------------------")
```

```python
    print(f"La flor es: IRIS {predicted_class.upper()}")
    print("-----------------------------")

    # Mostramos las probabilidades
    print("\nProbabilidades:")
    for i in range(len(labels)):
        print(f"{labels[i]}: {probs[i]*100:.2f}%")

except:
    # Si el usuario escribe mal
    print("Error: escribe solo números")
```

```
=== PREDICCIÓN DE FLOR IRIS ===
Introduce los valores:

-----------------------------
La flor es: IRIS SETOSA
-----------------------------

Probabilidades:
setosa: 97.69%
versicolor: 2.31%
virginica: 0.00%
```

GitHub

https://github.com/Erick-305/machine-learning.git